

Java Cheat Sheet — de Java 8 a Java 21

Referencia rápida de lo más usado en el día a día, versión por versión, con ejemplos. Las versiones **LTS** son 8, 11, 17 y 21.

Versión	Año	LTS	Titulares
8	2014	Sí	Lambdas, Streams, Optional, <code>java.time</code> , métodos <code>default</code>
9	2017		Módulos, <code>List.of()</code> , <code>takeWhile</code> , métodos privados en interfaces
10	2018		<code>var</code> (inferencia local), <code>copyOf</code> , <code>orElseThrow()</code>
11	2018	Sí	Métodos de <code>String</code> , <code>HttpClient</code> , <code>Files.readString</code> , <code>var</code> en lambdas
12-13	2019		<code>Collectors.teeing</code> , <code>String.transform</code> , previews de switch y text blocks
14	2020		Switch expressions (final), NPE útiles, preview de records
15	2020		Text blocks (final), preview de sealed
16	2021		Records (final), <code>instanceof</code> con patrón, <code>Stream.toList()</code>
17	2021	Sí	Sealed classes (final), <code>RandomGenerator</code> , base moderna estándar
18-20	2022-23		Web server simple, UTF-8 por defecto, previews de virtual threads
21	2023	Sí	Virtual threads , pattern matching en <code>switch</code> , record patterns, sequenced collections

Java 8 — el cambio más grande

Lambdas e interfaces funcionales

```
// Antes
Runnable r = new Runnable() {
    public void run() { System.out.println("hola"); }
};

// Ahora
Runnable r = () -> System.out.println("hola");

Comparator<Cliente> porNombre = (a, b) -> a.getNombre().compareTo(b.getNombre());
Comparator<Cliente> porEdadLuegoNombre =
    Comparator.comparingInt(Cliente::getEdad)
        .thenComparing(Cliente::getNombre)
        .reversed();
```

Interfaz	Firma	Uso típico
<code>Function<T,R></code>	<code>R apply(T t)</code>	<code>map()</code>
<code>BiFunction<T,U,R></code>	<code>R apply(T,U)</code>	<code>merge()</code> , <code>reduce()</code>
<code>Predicate<T></code>	<code>boolean test(T t)</code>	<code>filter()</code> , <code>removeIf()</code>
<code>Consumer<T></code>	<code>void accept(T t)</code>	<code>forEach()</code>
<code>Supplier<T></code>	<code>T get()</code>	<code>orElseGet()</code> , lazy
<code>UnaryOperator<T></code>	<code>T apply(T t)</code>	<code>replaceAll()</code>
<code>BinaryOperator<T></code>	<code>T apply(T,T)</code>	<code>reduce()</code>

```
Predicate<String> vacio = String::isEmpty;
Predicate<String> noVacio = vacio.negate(); // .and(...) .or(...)
Function<Integer,Integer> doble = x -> x * 2;
doble.andThen(x -> x + 1).apply(5); // 11
doble.compose(x -> x + 1).apply(5); // 12
```

Method references

```
Cliente::getNombre // método de instancia de un tipo arbitrario
System.out::println // método de instancia de un objeto concreto
Integer::parseInt // método estático
Cliente::new // constructor
```

Stream API

```
List<Cuenta> cuentas = ...;

List<String> resultado = cuentas.stream()
    .filter(c -> c.getSaldo().compareTo(BigDecimal.ZERO) > 0)
    .sorted(Comparator.comparing(Cuenta::getSaldo).reversed())
    .map(Cuenta::getNumero)
    .limit(10)
    .collect(Collectors.toList());
```

Operación	Tipo	Qué hace
<code>filter</code> , <code>map</code> , <code>flatMap</code> , <code>distinct</code> , <code>sorted</code> , <code>limit</code> , <code>skip</code> , <code>peek</code>	intermedia (lazy)	Arma el pipeline
<code>collect</code> , <code>forEach</code> , <code>reduce</code> , <code>count</code> , <code>min</code> , <code>max</code>	terminal	Dispara la ejecución
<code>anyMatch</code> , <code>allMatch</code> , <code>noneMatch</code> , <code>findFirst</code> , <code>findAny</code>	terminal (corto-circuito)	Devuelve apenas puede

```
// flatMap: aplanar colecciones anidadas
List<Movimiento> todos = clientes.stream()
    .flatMap(c -> c.getMovimientos().stream())
    .collect(Collectors.toList());

// reduce
BigDecimal total = movimientos.stream()
    .map(Movimiento::getImporte)
    .reduce(BigDecimal.ZERO, BigDecimal::add);

// Streams primitivos (sin boxing)
IntSummaryStatistics st = cuentas.stream().mapToInt(Cuenta::getEdad).summaryStatistics();
st.getMax(); st.getAverage(); st.getCount();

IntStream.rangeClosed(1, 5).sum(); // 15
Stream.of("a", "b").forEach(System.out::println);
Arrays.stream(array).boxed().toList(); // Java 16+
```

Collectors

```
Map<String, List<Cliente>> porSucursal = clientes.stream()
    .collect(Collectors.groupingBy(Cliente::getSucursal));

Map<String, Long> conteo = clientes.stream()
    .collect(Collectors.groupingBy(Cliente::getSucursal, Collectors.counting()));

Map<String, BigDecimal> sumaPorSucursal = clientes.stream()
    .collect(Collectors.groupingBy(Cliente::getSucursal,
        Collectors.reducing(BigDecimal.ZERO, Cliente::getSaldo, BigDecimal::add)));

Map<Boolean, List<Cliente>> particion = clientes.stream()
    .collect(Collectors.partitioningBy(c -> c.getEdad() >= 18));

Map<Long, Cliente> porId = clientes.stream()
    .collect(Collectors.toMap(Cliente::getId, Function.identity(),
        (viejo, nuevo) -> nuevo)); // 3er arg evita duplicate key

String csv = clientes.stream().map(Cliente::getNombre)
    .collect(Collectors.joining(", ", "[", "]"));
```

Trampa clásica: `Collectors.toMap` explota con `IllegalStateException` si hay claves repetidas y no pasás la función de merge. Y `groupingBy` no acepta claves `null`.

Optional

```
Optional<Cliente> opt = repo.findById(id);

cliente.map(Cliente::getEmail)
    .filter(e -> e.contains("@"))
    .orElse("sin-email");

opt.orElseGet(Cliente::new); // lazy, preferido frente a orElse(new ...)
opt.orElseThrow(() -> new NotFoundException(id));
opt.ifPresent(c -> log.info("Encontrado {}", c));
Optional.ofNullable(mapa.get(k));
```

Usá `Optional` como **tipo de retorno**, no como campo de entidad ni parámetro de método. Nunca `opt.get()` sin `isPresent()`.

Métodos default y static en interfaces

```
public interface Validador {
    boolean valida(String in);
    default Validador y(Validador otro) { return s -> valida(s) && otro.valida(s); }
    static Validador noNulo() { return Objects::nonNull; }
}
```

API de fechas `java.time` (reemplaza `Date` / `Calendar`)

```
LocalDate hoy = LocalDate.now();
LocalDate vto = hoy.plusMonths(1).withDayOfMonth(10);
LocalDateTime ts = LocalDateTime.of(2026, Month.MARCH, 5, 14, 30);
ZonedDateTime zdt = ZonedDateTime.now(ZoneId.of("America/Argentina/Cordoba"));
Instant instante = Instant.now(); // UTC, para timestamps

Period p = Period.between(nacimiento, hoy); // años/meses/días
long dias = ChronoUnit.DAYS.between(desde, hasta);
Duration d = Duration.ofMinutes(90);

DateTimeFormatter fmt = DateTimeFormatter.ofPattern("dd/MM/yyyy");
String s = hoy.format(fmt);
LocalDate parsed = LocalDate.parse("05/03/2026", fmt);
```

Todos los tipos son **inmutables y thread-safe** (a diferencia de `SimpleDateFormat`).

Novedades en `Map` y colecciones

```
mapa.getOrDefault(k, 0);
mapa.putIfAbsent(k, v);
mapa.computeIfAbsent(k, key -> new ArrayList<>()).add(item);
mapa.merge(k, 1, Integer::sum); // contador
mapa.forEach((k, v) -> ...);
lista.removeIf(c -> c.getSaldo() == null);
lista.replaceAll(String::trim);
lista.sort(Comparator.naturalOrder());
```

CompletableFuture

```
CompletableFuture<Cliente> f = CompletableFuture.supplyAsync(() -> repo.buscar(id), executor);

f.thenApply(Cliente::getSaldo)
 .thenCombine(otroFuture, (a, b) -> a.add(b))
 .exceptionally(ex -> BigDecimal.ZERO)
 .thenAccept(System.out::println);

CompletableFuture.allOf(f1, f2, f3).join();
```

Java 9

```
// Colecciones inmutables por factory
List<String> l = List.of("a", "b");
Set<Integer> s = Set.of(1, 2, 3);
Map<String,Integer> m = Map.of("a", 1, "b", 2);
Map<String,Integer> m2 = Map.ofEntries(Map.entry("a",1), Map.entry("b",2));
// ¡Lanzan UnsupportedOperationException al modificar y no aceptan null!

// Streams
Stream.iterate(1, i -> i <= 100, i -> i * 2).forEach(...); // con condición
stream.takeWhile(x -> x < 10); // toma hasta que falle
stream.dropWhile(x -> x < 10); // descarta hasta que falle
Stream.ofNullable(puedeSerNull);

// Optional
opt.ifPresentOrElse(this::procesar, this::registrarFaltante);
opt.or(() -> Optional.of(porDefecto));
opt.stream(); // para encadenar con flatMap

// Métodos privados en interfaces
interface Servicio {
    default void a() { comun(); }
    default void b() { comun(); }
    private void comun() { ... }
}

// try-with-resources con variable ya efectivamente final
try (conexionExistente) { ... }
```

Módulos (JPMS) — `module-info.java`. En la práctica la mayoría de los proyectos Spring Boot no lo usan, pero conviene reconocerlo:

```
module com.banco.clientes {
    requires spring.core;
    exports com.banco.clientes.api;
}
```

Java 10

```

var lista = new ArrayList<Cliente>();           // inferencia SÓLO en variables locales
for (var e : mapa.entrySet()) { ... }         // también en el for-each
// NO sirve para campos, parámetros ni retornos. No uses var si pierde legibilidad.

List<String> copia = List.copyOf(original);     // copia inmutable
stream.collect(Collectors.toUnmodifiableList());
optional.orElseThrow();                       // = get() pero con nombre honesto

```

Java 11 (LTS)

```

// String
" ".isBlank();           // true
" hola ".strip();        // Unicode-aware, mejor que trim()
"ab".repeat(3);          // "ababab"
texto.lines().filter(l -> !l.isBlank()).toList();

// Files
String contenido = Files.readString(Path.of("in.txt"));
Files.writeString(Path.of("out.txt"), contenido);

// Predicate.not
lista.stream().filter(Predicate.not(String::isBlank)).toList();

// var en parámetros de lambda (útil solo para poner anotaciones)
(@NonNull var a, @NonNull var b) -> a + b

// toArray más limpio
String[] arr = lista.toArray(String[]::new);

// HttpClient estándar (reemplaza HttpURLConnection)
HttpClient client = HttpClient.newHttpClient();
HttpRequest req = HttpRequest.newBuilder(URI.create("https://api/x"))
    .header("Content-Type", "application/json")
    .POST(HttpRequest.BodyPublishers.ofString(json))
    .timeout(Duration.ofSeconds(5))
    .build();
HttpResponse<String> res = client.send(req, HttpResponse.BodyHandlers.ofString());
client.sendAsync(req, BodyHandlers.ofString()).thenApply(HttpResponse::body);

```

`java Hola.java` — ejecuta un archivo único sin compilar aparte.

Java 12 y 13

```

// Collectors.teeing (12): dos colectores y un merge
record Resumen(long cantidad, BigDecimal total) {}
Resumen r = movimientos.stream().collect(Collectors.teeing(
    Collectors.counting(),
    Collectors.reducing(BigDecimal.ZERO, Movimiento::getImporte, BigDecimal::add),
    Resumen::new));

// String (12)
"texto".indent(4);
String id = "abc".transform(s -> s.toUpperCase() + "-01");

```

En 12/13 llegan en *preview* las switch expressions y los text blocks (finales en 14 y 15).

Java 14

Switch expressions (final)

```

// Devuelve valor, sin fall-through, con -> y yield
String tipo = switch (codigo) {
    case "CA", "CC" -> "Cuenta";
    case "PF"      -> "Plazo fijo";
    default        -> {
        log.warn("desconocido {}", codigo);
        yield "Otro";
    }
};

```

El compilador exige **exhaustividad** cuando se usa como expresión (con `enum` completo o `default`).

instanceof con patrón (preview en 14, final en 16)

```

// Antes
if (o instanceof Cliente) { Cliente c = (Cliente) o; usar(c.getNombre()); }

// Ahora
if (o instanceof Cliente c && c.getEdad() > 18) { usar(c.getNombre()); }

```

Helpful NullPointerExceptions

Activado por defecto desde 15: el mensaje indica **qué** fue null (`Cannot invoke "Cuenta.getSaldo()" because the return value of "Cliente.getCuenta()" is null`).

Java 15 — Text blocks (final)

```
String sql = """
    SELECT c.id, c.nombre, SUM(m.importe) AS total
    FROM cliente c
    JOIN movimiento m ON m.cliente_id = c.id
    WHERE c.estado = ?
    GROUP BY c.id, c.nombre
    """;

String json = """
    {"id": %d, "nombre": "%s"}
    """.formatted(id, nombre);
```

La indentación común se elimina sola. `\` al final une líneas; `\s` preserva un espacio.

Java 16 — Records (final)

```
public record ClienteDTO(Long id, String nombre, BigDecimal saldo) {

    // constructor compacto: validaciones / normalización
    public ClienteDTO {
        Objects.requireNonNull(nombre);
        saldo = saldo == null ? BigDecimal.ZERO : saldo;
    }

    public static ClienteDTO de(Cliente c) { return new ClienteDTO(c.getId(), c.getNombre(), c.getSaldo()); }
}
```

Genera automáticamente constructor, accessors (`id()`, no `getId()`), `equals`, `hashCode` y `toString`. Es **final e immutable**: ideal para DTOs, value objects, claves de mapa y respuestas de API.

```
List<String> nombres = stream.map(Cliente::getNombre).toList(); // immutable, más corto que collect()
stream.mapMulti(...); // 16: alternativa eficiente a flatMap
```

Java 17 (LTS) — Sealed classes

```
public sealed interface Pago permits PagoTarjeta, PagoTransferencia, PagoEfectivo {}

public record PagoTarjeta(String numero, int cuotas) implements Pago {}
public record PagoTransferencia(String cbu) implements Pago {}
public final class PagoEfectivo implements Pago {}
// Las subclases deben ser final, sealed o non-sealed
```

Cerrar la jerarquía permite al compilador verificar exhaustividad en `switch` (sin `default`). Combinado con records se logra el patrón *algebraic data type*.

```
RandomGenerator rnd = RandomGenerator.of("L64X128MixRandom"); // nueva API de aleatorios
```

Java 17 es la línea base habitual de Spring Boot 3.x.

Java 18, 19 y 20

```
// 18: UTF-8 es el charset por defecto en toda la plataforma
// 18: servidor web mínimo para pruebas
jwebserver -p 8000

// 19/20: virtual threads, record patterns y structured concurrency en preview
```

Java 21 (LTS) — la versión a la que conviene apuntar

Virtual threads

```
// Un hilo virtual por tarea: baratísimos, se pueden crear millones
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (var id : ids) {
        executor.submit(() -> servicio.procesar(id)); // bloquear acá NO consume un hilo del SO
    }
} // close() espera a que terminen todas

Thread.startVirtualThread(() -> log.info("corriendo"));
```

Ideal para cargas **IO-bound** (JDBC, HTTP, colas): el código bloqueante clásico escala como si fuera reactivo. Cuidados: evitar `synchronized` en secciones

bloqueantes (usar `ReentrantLock`) y no usar pooling de virtual threads.

Pattern matching para `switch` + record patterns

```
String describir(Pago p) {
    return switch (p) {
        case PagoTarjeta(String nro, int cuotas) when cuotas > 1
            -> "Tarjeta %s en %d cuotas".formatted(nro, cuotas);
        case PagoTarjeta t
            -> "Tarjeta " + t.numero();
        case PagoTransferencia(String cbu) -> "Transferencia a " + cbu;
        case PagoEfectivo e
            -> "Efectivo";
        case null
            -> "Sin pago";
        // sin default: el compilador sabe que es exhaustivo (sealed)
    };
}
```

`when` reemplaza a los viejos guards con `if`. Los record patterns permiten **desestructurar** (incluso anidado: `case Linea(Punto(int x1, int y1), Punto p2) -> ...`).

Sequenced collections

```
SequencedCollection<String> c = new ArrayList<>(List.of("a", "b", "c"));
c.getFirst(); c.getLast();
c.addFirst("z"); c.removeLast();
c.reversed(); // vista invertida

SequencedMap<String,Integer> m = new LinkedHashMap<>();
m.firstEntry(); m.lastEntry(); m.putFirst("a", 1);
```

Unifica por fin `List`, `Deque`, `LinkedHashSet` y `LinkedHashMap` bajo una API de orden común.

Otros de 21

- `String templates` y *unnamed patterns* (`case Punto(int x, _)`) llegaron como **preview** (los templates fueron luego retirados: no los uses en producción).
- Generational ZGC**: pausas de GC en torno al milisegundo.
- `Math.clamp`, mejoras de `KeyEncapsulation`, API de `Vector` aún en incubación.

Tabla de migración rápida

En vez de...	Usá...	Desde
<code>new Date()</code> , <code>SimpleDateFormat</code>	<code>LocalDate</code> , <code>DateTimeFormatter</code>	8
<code>if (x != null)</code> en cascada	<code>Optional</code> encadenado	8
Bucles <code>for</code> con acumuladores	<code>Streams</code> + <code>Collectors</code>	8
<code>Arrays.asList(...)</code>	<code>List.of(...)</code>	9
<code>collect(Collectors.toList())</code>	<code>.toList()</code>	16
Clases DTO con Lombok <code>@Value</code>	<code>record</code>	16
Cast tras <code>instanceof</code>	<code>instanceof Tipo t</code>	16
Jerarquías abiertas + <code>default</code>	<code>sealed</code> + <code>switch</code> exhaustivo	17
Concatenar SQL/JSON con <code>+</code>	text blocks <code>"""</code>	15
Pools de hilos gigantes para IO	virtual threads	21
<code>URLConnection</code>	<code>HttpClient</code>	11